JavaScript Async Common Mistakes

Back to JS page

Table of Contents

Mistake 1: Forgetting await

Forgetting `await` returns a Promise object instead of the actual value:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 1: Forgetting await</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
async function getValue() {
  return "Hello World";
}

// WRONG: missing await - returns a Promise object
let wrong = getValue();
document.getElementById("demo1").innerHTML = "Without await: " + wrong;

// CORRECT: with await - returns the actual value
async function correctVersion() {
  let correct = await getValue();
  document.getElementById("demo2").innerHTML = "With await: " + correct;
}
correctVersion();
</script>

</body>
</html>
```

Example 1

Result View Example

Mistake 2: Using await Outside an Async Function

`await` can only be used inside `async` functions:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 2: await Outside Async Function</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// WRONG: await cannot be used at the top level (in older JS)
// let result = await getValue(); // SyntaxError

// CORRECT: wrap in async function
async function getData() {
  let promise = new Promise(function(resolve) {
    setTimeout(function() { resolve("Data loaded"); }, 1000);
  });
  let result = await promise;
  document.getElementById("demo1").innerHTML = "await works inside async: " + result;
}

getData();

// Alternative: using .then() outside async function
let promise2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Data via .then()"); }, 1500);
});
promise2.then(function(value) {
  document.getElementById("demo2").innerHTML = "Using .then() outside async: " + value;
});
</script>

</body>
</html>
```

Example 2

Result View Example

Mistake 3: Assuming fetch() Fails on HTTP Errors

`fetch()` only rejects on network errors, not HTTP errors (like 404 or 500). You must check `response.ok`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 3: fetch() and HTTP Errors</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
async function fetchWithoutCheck() {
  // fetch() does NOT throw on HTTP errors like 404
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');
  document.getElementById("demo1").innerHTML =
    "fetch() did NOT throw! Status: " + response.status + " " + response.statusText;
}
fetchWithoutCheck();

async function fetchWithCheck() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users/invalid');
  if (!response.ok) {
    document.getElementById("demo2").innerHTML =
      "Error caught: HTTP " + response.status + " (" + response.statusText + ")";
    return;
  }
  const data = await response.json();
  document.getElementById("demo2").innerHTML = "Name: " + data.name;
}
fetchWithCheck();
</script>

</body>
</html>
```

Example 3

Result View Example

Mistake 4: Blocking the Event Loop

Long-running synchronous operations block async code from executing:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 4: Blocking the Event Loop</h4>
<p id="demo"></p>

<script>
// This timeout should run after 0ms
setTimeout(function() {
  document.getElementById("demo").innerHTML += "2. setTimeout (delayed!)<br>";
}, 0);

// Blocking operation for 2 seconds
let start = Date.now();
while (Date.now() < start + 2000) {
  // Blocks the event loop
}

document.getElementById("demo").innerHTML += "1. After blocking loop (2 seconds)<br>";
</script>

</body>
</html>
```

Example 4

Result View Example

Mistake 5: Running Independent Tasks One After Another

Independent tasks should run in parallel, not sequentially:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Async Common Mistakes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Mistake 5: Sequential Independent Tasks</h4>
<p id="demo"></p>

<script>
async function demonstrate() {
  // WRONG: running independent tasks sequentially (2 seconds total)
  let task1 = await new Promise(function(resolve) {
    setTimeout(function() { resolve("Task 1 done"); }, 1000);
  });
  let task2 = await new Promise(function(resolve) {
    setTimeout(function() { resolve("Task 2 done"); }, 1000);
  });

  document.getElementById("demo").innerHTML += "WRONG (sequential): ~2 seconds<br>";
  document.getElementById("demo").innerHTML += task1 + "<br>" + task2 + "<br><br>";

  // CORRECT: running independent tasks in parallel (~1 second)
  let start = Date.now();
  let [result1, result2] = await Promise.all([
    new Promise(function(resolve) {
      setTimeout(function() { resolve("Task 1 done"); }, 1000);
    }),
    new Promise(function(resolve) {
      setTimeout(function() { resolve("Task 2 done"); }, 1000);
    })
  ]);

  document.getElementById("demo").innerHTML += "CORRECT (parallel): ~1 second<br>";
  document.getElementById("demo").innerHTML += result1 + "<br>" + result2;
}

demonstrate();
</script>

</body>
</html>
```

Example 5

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Async Common Mistakes